Question 1: What is the Django Template Engine? Explain the differences between Django Template Tags and Template Filters with suitable examples.

Answer

The Django Template Engine is used to generate dynamic HTML pages by combining HTML templates with data sent from views. It supports variables, tags, and filters.

Template Tags

Template tags control the logic and structure of a template. They are written inside {% %}.

```
{% if user.is_authenticated %}
    <p>Welcome!</p>
{% endif %}

{% for item in items %}
    <p>{{ item }}</p>
{% endfor %}
```

Template Filters

Template filters modify or format variable values. They are written using the pipe (|) symbol.

```
{{ name|upper }}
{{ date|date:"d M Y" }}
{{ description|truncatewords:10 }}
```

Question 2: What is Django Middleware? Explain the differences between Built-in Middleware and Custom Middleware. Also describe the middleware execution order.

Answer

Django Middleware is a component that processes requests before they reach a view and responses before they are sent to the client.

Built-in Middleware

Built-in Middleware is provided by Django to handle common tasks.

- SecurityMiddleware: provides security-related protections.
- SessionMiddleware: manages user sessions.
- CsrfViewMiddleware: protects against CSRF attacks.
- AuthenticationMiddleware: identifies the current user.

Custom Middleware

Custom Middleware is created by developers to perform application-specific tasks, such as logging requests or adding response headers.

```
class CustomMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
      response["X-App"] = "My App"
        return response
```

Built-in Middleware	**Custom Middleware**
Handles common tasks.	Handles specific application requirements.
Provided by Django.	Created by developers.
Example : SessionMiddleware	Example: custom logging middleware

Middleware Execution Order

During a request, middleware executes from top to bottom according to the MIDDLEWARE list in settings.py. During response processing, it executes in reverse order, from bottom to top.

```
Request:  Middleware 1 → Middleware 2 → View
Response: Middleware 1 ← Middleware 2 ← View
```

Question 3: What is a Django ModelForm? Explain how to create a ModelForm and handle HTML form submissions using GET and POST requests.

Answer

A Django ModelForm is a form automatically generated from a model. It simplifies form creation, validation, and saving data to the database.

First, create a model:

```
# models.py
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
```

Create a ModelForm:

```
# forms.py
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ["name", "email"]
```

Handle GET and POST requests in a view:

```
# views.py
def student_create(request):
    if request.method == "POST":
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("home")
    else:
        form = StudentForm()

    return render(request, "student_form.html", {"form": form})
```

GET is used to display an empty form. POST is used to submit form data. The is_valid() method checks the submitted data, and save() stores valid data in the database.

Display the form in HTML:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

Question 4: What is the difference between Static Files and Media Files in Django? Explain how to configure and handle image uploads using ImageField, MEDIA_URL, and MEDIA_ROOT.

Answer

Static files are files used by the website, such as CSS, JavaScript, and logos. Media files are uploaded by users, such as profile pictures and documents.

Static Files	Media Files
Usually provided by developers.	Uploaded by users.
Examples: CSS, JS, images.	Examples: profile photos, documents.
Configured using STATIC_URL and STATIC_ROOT.	Configured using MEDIA_URL and MEDIA_ROOT.

Static Files	Media Files
Usually provided by developers.	Uploaded by users.
Configured using STATIC_URL and STATIC_ROOT.	Configured using MEDIA_URL and MEDIA_ROOT.
Example: CSS, Javascript etc.	Example : profile picture, any documents etc.

Configure media settings in settings.py:

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Add an ImageField to a model:

```
class Profile(models.Model):
    photo = models.ImageField(upload_to="profiles/")
```

Install Pillow, which is required by ImageField:

```
pip install Pillow
```

Handle the uploaded file in a view:

```
form = ProfileForm(request.POST, request.FILES)
if form.is_valid():
    form.save()
```

The HTML form must use multipart/form-data:

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

MEDIA_ROOT specifies the directory where uploaded files are stored, while MEDIA_URL specifies the URL used to access them. In production, media files should be served through a web server or storage service.

Question 5: Explain Django Authentication and Session Middleware. How do Login, Logout, and Session Management work in Django?

Answer

Django Authentication is a built-in system for managing users, passwords, login, and permissions.

SessionMiddleware manages session data and makes it available through request.session. AuthenticationMiddleware uses session information to identify the logged-in user and makes the user available through request.user. AuthenticationMiddleware must be placed after SessionMiddleware.

Login

Django uses authenticate() to verify user credentials and login() to create an authenticated session.

```
from django.contrib.auth import authenticate, login

user = authenticate(request, username=username, password=password)
if user is not None:
    login(request, user)
```

Logout

The logout() function logs out the current user and clears the session data.

```
from django.contrib.auth import logout

def user_logout(request):
    logout(request)
    return redirect("login")
```

Session Management

Sessions allow Django to store user-specific data across multiple requests. For example:

```
request.session["cart_count"] = 3
cart_count = request.session.get("cart_count", 0)
```

By default, Django stores session data on the server and keeps a session ID in the browser cookie. This allows the user to remain logged in across requests.